BEAT THE BOOKIE

A PREPRINT

Group Name: `Group I`
Department of Computer Science
University College London
London, WC1E 6BT

December 16, 2025

1 Introduction

This project formulates the prediction of English Premier League match outcomes as a supervised learning task. Our approach combines classical Poisson regression with gradient boosting classifiers (CatBoost, XGBoost) in a stacking ensemble. The ensemble achieves 56.21% accuracy on the held-out test set, exceeding the 53% [1] bookie benchmark.

2 Data Transformation & Exploration

We enriched the base dataset with external sources [see Section 3.2], including xG metrics, Elo ratings, and in-match statistics (possession, shots, fouls) [2, 3, 4]. Some advanced features are unavailable for older fixtures; we retain them where available since recent matches carry greater predictive relevance. Feature availability distribution is shown in Figure 2.

2.1 Data Acquisition and Preprocessing

To enrich the baseline training dataset, we collected additional match-level data from external sources using scripted API queries. In-game statistics, such as possession, shots on target, corners, and disciplinary events were retrieved via the SportDB (Flashscore) API [5], while Elo rating data were sourced from ClubElo [6].

The external features were integrated with the official competition data by aligning on three key columns: match date, home team, and away team. The dataset was filtered to retain only matches present in the original training set, preventing data leakage from unseen fixtures. The final unified `DataFrame` was then retained for downstream processing.

2.2 Exploratory Data Analysis

Correlation heatmaps identified collinearity across feature groups; redundant features were removed or aggregated, and interaction features were engineered. Categorical features were encoded consistently across training and test data.

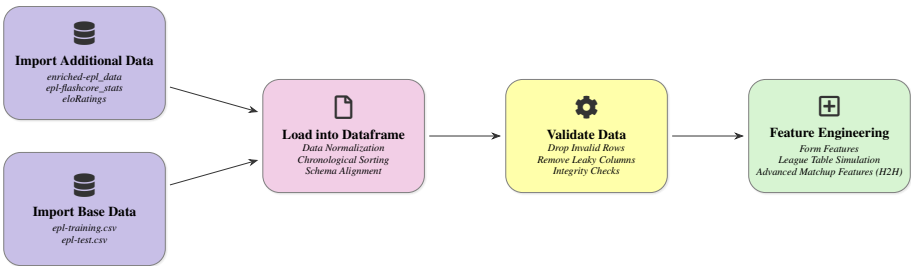


Figure 1: Overview of the data preprocessing pipeline.

3 Methodology Overview

Our methodology combines statistical and machine learning approaches within a stacking ensemble. We first enrich the base dataset with external sources (Elo ratings, xG metrics, in-match statistics), then engineer over 70 features capturing team strength, recent form, and head-to-head trends. Three base models: a time-aware Poisson regression, CatBoost, and XGBoost. These generate probabilistic predictions that are combined via a logistic regression meta-learner. All training and validation follows strict chronological ordering to prevent data leakage.

3.1 Background Reading

Prior work [7, 8, 9] highlights the value of engineered features (Elo ratings, xG, rolling form metrics) and demonstrates gradient boosting methods (XGBoost [10], CatBoost [11]) outperform traditional classifiers on structured football data due to superior handling of nonlinear feature interactions. Guided by these insights, we incorporated both engineered features and boosting-based models into our pipeline.

3.2 Additional Data Sources

To overcome the limitations of the base dataset, we integrated several external sources:

- **ClubElo.com** [6]: Historical Elo ratings, a continuous indicator of team strength.
- **xG Data**: Expected goals statistics for both teams, indicating shot quality.
- **League Tables**: Points, wins, goal difference, and rank from current season.
- **Flashscore API via SportDB** [5]: Detailed match stats (e.g., possession, shots, fouls) for all Premier League fixtures since 2000.

3.3 Experimental Process

Initial experiments used regression to predict goal counts (Random Forest, XGBoost regressors), but this proved unstable; football scores are discrete and low, so small prediction differences (e.g., 1.6 vs 1.4 goals) frequently flipped inferred outcomes. Single-feature approaches using Elo difference or xG differential suffered the same threshold instability. Random Forest classifiers exhibited severe overfitting on training data despite regularisation attempts, failing to generalise across seasons.

Reformulating as direct H/D/A classification with gradient boosting resolved these issues, showing consistent improvements when supplied with engineered features. The final architecture combines complementary base models (Poisson regression, CatBoost, XGBoost) in a stacking ensemble.

3.4 Validation Strategy and Key Lessons

Early on, we mistakenly used random shuffling when splitting train and validation data (`shuffle=True`), which violated temporal order and led to substantial data leakage. Model performance during cross-validation was artificially inflated due to exposure to future data.

We corrected this by implementing strict time-based splits. All training data preceded validation data chronologically, simulating a realistic prediction environment. We also ensured that rolling features were based only on prior matches by using `.shift(1)` before applying rolling windows.

To represent team form across different time scales, we computed sliding-window aggregates over 3, 5, and 10 previous games. These rolling features captured short- and medium-term trends, and were particularly helpful for variables like Elo, xG, goal difference, and win/loss streaks. In later stages, we experimented with additional time-weighted schemes that emphasised more recent matches, a light form of exponential decay, to further refine trend modelling. We found this rolling window approach consistently outperformed exponential time decay [12].

4 Model Training & Validation

4.1 Training Objective and Problem Formulation

The final modelling objective was defined as a direct multi-class classification problem, predicting the full-time result (Home win, Draw, Away win). Early experiments attempted to predict exact goal counts using regression-based

approaches; however, this formulation proved unstable due to the low-scoring and discrete nature of football outcomes. Small numerical errors in predicted goals frequently resulted in large categorical misclassification.

Reformulating the task as direct outcome classification significantly improved predictive stability and enabled the models to learn decision boundaries aligned with the evaluation objective. Furthermore, this formulation naturally supports probabilistic outputs, which are critical for ensemble stacking and downstream calibration.

4.2 Feature Preparation for Training

Feature preparation was designed to ensure temporal validity, robust generalisation, and compatibility between training and deployment conditions. All engineered features were computed exclusively from information available prior to match kick-off, with grouped rolling windows combined with a `.shift(1)` operation to prevent any leakage of contemporaneous or future match information.

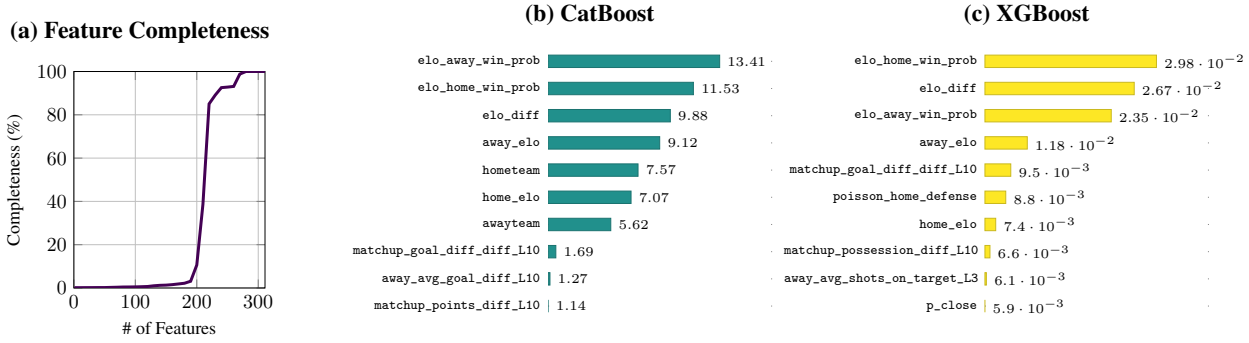


Figure 2: (a) Data completeness distribution showing many features at 100% with a tail of sparse features. (b-c) Top 10 feature importances for CatBoost and XGBoost classifiers.

Note: The x axis in (b-c) represents the outputted `Importance` metrics from the models `get_feature_importance()` functions. This varies from model to model hence the variance in x-axis scaling.

Feature construction was performed in the *Advanced Features* stage, resulting in over 70 core features and more than 300 derived features after interaction and differential expansion. These features were grouped into categories including:

- **Team Strength Metrics:** Elo ratings from ClubElo, league ranks, points per game, and goal difference.
- **Recent Form:** Rolling averages of recent match outcomes (Win/Draw/Loss encoded numerically), for 3, 5, and 10-game windows.
- **Performance Indicators:** Shots on target, possession percentages, fouls, and xG data (expected goals), both absolute and relative.
- **Shots on Target Ratio (STR):** STR from rolling averages.
- **Head-to-Head (H2H) Trends:** Aggregated past outcomes and goals in recent H2H matchups. Populated for 100% of matches.
- **Strength Differential Features:** Home team minus away team differences across several metrics.
- **EPL Table Constructor:** A synthetic EPL table of points up to the specified point in the season. Populated for 96.8% of matches.
- **Draw-Specific Indicators:** Binary and composite features capturing mid-table positioning, tactical stalemates (balanced possession with low attacking output), and late-season motivational lulls, conditions historically correlated with drawn outcomes.

Certain advanced statistics (e.g., long passing metrics) were only available for a limited subset of matches (Figure 2a). Rather than discarding these features, we retained them deliberately. Older seasons are less representative of contemporary playing styles and tactical structures, and enforcing full availability would bias the model toward outdated dynamics.

Empirically, retaining partially missing high-signal features improved validation performance on recent splits compared to enforcing full availability, confirming older seasons are less representative of contemporary dynamics. Feature importance comparisons across models are shown in Figure 2b, illustrating that the models consistently prioritised robust, temporally stable features over sparse historical signals who's contribution is deliberately regularised and secondary to dynamic performance features.

4.3 Auxiliary Models

We introduce an auxiliary model that predicts match competitiveness rather than outcomes directly. It estimates the probability of a closely contested fixture, capturing structural patterns difficult to express through static features. This probabilistic signal is injected into downstream classifiers during training and inference.

4.3.1 Match Closeness Model

A CatBoost binary classifier trained to predict whether a match will be decided by one goal or fewer ($|GD| \leq 1$). Approximately 70–75% of Premier League fixtures fall within this margin, and such matches exhibit higher draw rates and reduced predictability from traditional strength differentials.

Instead of outputting a class label, the model produces a probability score, `p_close`, representing the likelihood of a tightly contested fixture. The classifier learns to associate pre-match feature patterns with match tightness, for example, similar Elo ratings yield higher `p_close` values, while mismatches between top-six sides and relegation candidates produce lower values.

The model is configured as follows:

- **Target Definition:** Binary label where `is_close = 1` if the final goal difference was ≤ 1. Labels are derived from historical results; the model receives only pre-match features as input.
- **Feature Set:** Identical to the primary classifiers (rolling averages, ELO differentials, xG metrics, etc.).
- **Leakage Prevention:** Out-of-fold predictions via `TimeSeriesSplit` for training samples (see Section 4.6.3).
- **Integration:** The output probability `p_close` is appended to training, validation and test data before training CatBoost and XGBoost. Unlike conventional engineered features, this is a learned representation derived from a trained model.

4.4 Base Models

Three base models were trained, spanning two complementary modelling paradigms: statistical goal-based estimation and machine learning outcome classification. Each model was selected to contribute a distinct inductive bias and error profile, maximising ensemble diversity and reducing correlated failures in downstream stacking.

4.4.1 Time-Aware Poisson Model [13]

We implemented a dynamic, time-aware variation of the classical Poisson regression. Unlike static implementations that estimate fixed team strengths over a season, this model calculates match-specific parameters using a rolling window of the last 5 matches (`L5`).

For a match where Home Team i hosts Away Team j, the probability of the home team scoring exactly k goals is modelled as:

$$P(X = k) = \frac{\lambda_{i,j}^{k} e^{-\lambda_{i,j}}}{k!}$$

Crucially, the expected goal rate $\lambda_{i,j}$ is derived specifically for this match-up:

$$\lambda_{i,j} = \mu \times \text{Att}_i \times \text{Def}_j \times \gamma$$

where μ is the league-wide average goal rate, Att_i is the rolling attacking strength of the home team, Def_j is the rolling defensive weakness of the away team, and γ is the league-wide home advantage factor.

The model outputs probabilities for all potential scorelines to derive the final $P(Home)$, $P(Draw)$, and $P(Away)$ vectors. Strictly historical data (using `.shift(1)`) was used to prevent data leakage.

The primary role of the Poisson model within the ensemble is to provide a statistically grounded, low-variance baseline rooted in football domain theory. While less expressive than boosting-based classifiers, its probabilistic structure offers a complementary signal that is largely uncorrelated with non-linear tree ensembles, particularly in matches with clear strength asymmetries.

4.4.2 CatBoost Classifier [11]

CatBoost was selected as the primary machine learning classifier due to strong performance on structured tabular data and native handling of high-cardinality categorical features, specifically the team identifiers (`hometeam`, `awayteam`).

To prevent the model from memorising team identities rather than learning form-based patterns and encourage generalisation across seasons, we applied a regularisation strategy:

- **Reduced Categorical Importance:** We restricted `one_hot_max_size=2` and increased `bagging_temperature=0.5`, forcing the model to rely more heavily on engineered features (e.g., `matchup_xg_diff`) rather than team names.
- **Loss Function:** The model was trained using `Multi-class` log-loss with class weights balanced to the training set distribution.
- **Hyperparameters:** A depth of 5 and learning rate of 0.03 were used with 1,000 iterations to ensure stable convergence without overfitting.

This configuration prioritised learning from dynamic performance indicators and match-up context rather than static team identity, improving robustness under temporal validation.

4.4.3 XGBoost Classifier [10]

XGBoost classifier was introduced to provide an alternative gradient-boosting signal. This model was distinctively configured to address the class imbalance inherent in football data, where draws are often under-predicted.

- **Cost-Sensitive Learning:** We implemented a custom "Draw Boost" weighting strategy, where the sample weight for draw outcomes was multiplied by a factor of 1.3. This penalised the model more heavily for missing draws compared to home or away wins. Although approximately 30% of matches in the dataset resulted in draws, unweighted models consistently under-predicted this class. The draw-boost weighting corrected this imbalance by increasing recall for drawn outcomes without materially degrading overall accuracy.
- **Regularisation:** To further reduce variance, strong regularisation was applied, including L1 (`alpha=2.0`) and L2 (`lambda=10.0`) penalties, alongside a reduced tree depth of 4.
- **Encoding:** Unlike CatBoost, categorical features were Label Encoded, providing a different structural representation of team identities to the ensemble.

4.5 Ensemble Stacking (Meta-Learner)

To synthesise the diverse signals from the base models, we employed a **Stacking Generalisation** (Stacking) approach. This technique involves training a meta-learner to optimally combine the probabilistic outputs of the base classifiers, effectively learning which model is most reliable for specific match contexts.

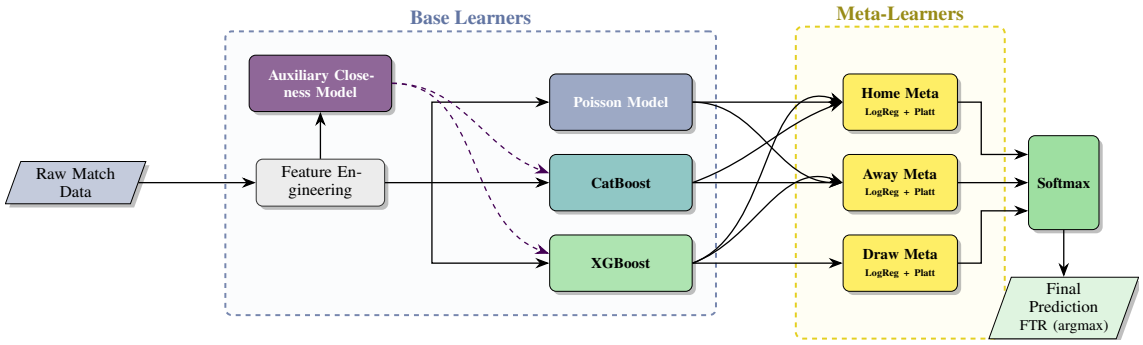


Figure 3: Architecture of the Hybrid Ensemble Model. Feature engineering feeds into three distinct base models, whose probabilistic outputs are concatenated into a meta-vector for the final Logistic Regression meta-learners.

4.5.1 Meta-Feature Construction

The input to the meta-learner was constructed by horizontally concatenating the class probability vectors from the three active base models:

1. **Time-Aware Poisson Model (sec. 4.4.1):** 3 features (P_{Home}, P_{Draw}, P_{Away}) — deterministic, no OOF required
2. **CatBoost Classifier (sec. 4.4.2):** 3 features (P_{Home}, P_{Draw}, P_{Away}) — OOF predictions

3. **XGBoost Classifier (sec. 4.4.3):** 3 features (P_{Home}, P_{Draw}, P_{Away}) — OOF predictions

This resulted in a dense, 9-dimensional meta-feature vector for each match. CatBoost and XGBoost predictions for training samples were generated via out-of-fold inference (see Section 4.6.3) to prevent the meta-learner from training on optimistically biased predictions.

4.5.2 Meta-Learner Configuration

A `LogisticRegression` classifier was selected as the meta-learner due to its interpretability and naturally calibrated probability outputs via softmax.

- **Regularisation:** Grid search over $C \in \{0.01, 0.05, 0.1, 0.3, 0.5, 1.0, 2.0\}$, selecting $C = 0.3$ as optimal.
- **Class Balancing:** `class_weight='balanced'` to maintain sensitivity to the minority Draw class.
- **Validation Strategy:** The meta-learner was trained on OOF predictions and evaluated on validation/test sets using full model predictions.

A simple linear meta-learner was preferred over tree-based alternatives to reduce overfitting risk on the compact 9-feature space. Tree-based meta-learners could memorise training patterns in such a low-dimensional input, whereas logistic regression provides smooth probability calibration with fewer parameters.

4.5.3 Final Prediction Generation

Multinomial logistic regression was used for final prediction generation. It consists of three parallel linear classifiers; one for each outcome class (Home, Draw, Away). Each classifier learns its own set of 9 weights over the meta-features, resulting in a coefficient matrix of shape (3×9) with 27 total parameters.

For a match with stacked feature vector $\mathbf{x}$, each classifier computes a logit score:

$$z_{\text{Home}} = b_H + \sum_i w_{Hi} \cdot x_i \qquad z_{\text{Draw}} = b_D + \sum_i w_{Di} \cdot x_i \qquad z_{\text{Away}} = b_A + \sum_i w_{Ai} \cdot x_i$$

The three logits are then compared via `softmax` to produce calibrated probabilities:

$$P(\text{class} = k \mid \mathbf{x}) = \frac{\exp(z_k)}{\sum_{j \in \{H,D,A\}} \exp(z_j)}$$

The final Full Time Result (`ftr`) is the class with highest probability:

$$\hat{y} = \arg \max_{k \in \{H,D,A\}} P(\text{class} = k \mid \mathbf{x})$$

This structure allows each outcome to learn different weightings of base model predictions—for instance, the Draw classifier may weight CatBoost's Home probability heavier than Poisson's outputs due to greater reliability.

4.5.4 Draw Prediction Override

Analysis of the learned coefficients (Figure 4) revealed that for Draw prediction, the meta-learner assigned disproportionate weight to CatBoost's draw probability despite CatBoost exhibiting systematic bias toward home/away outcomes. Examination of individual confusion matrices (Figure 7) showed XGBoost achieved superior draw recall compared to CatBoost, owing to its explicit draw-boost weighting strategy.

To exploit this, we modified the meta-learner's Draw classifier to use only XGBoost's probability outputs (P_H, P_D, P_A) as input features, rather than the full 9-dimensional vector from all three base models. This effectively removes CatBoost and Poisson's influence on draw predictions while preserving the standard 9-feature ensemble for Home/Away classification.

4.6 Validation & Evaluation Strategy

To simulate real-world deployment conditions and prevent look-ahead bias, we adopted a strict chronological evaluation protocol rather than randomised cross-validation. Random shuffling was explicitly avoided, as it violates the temporal structure of football data and can lead to overly optimistic performance estimates through exposure to future information.

This time-aware strategy respects the evolving nature of team form and ensures that all models are trained exclusively on past matches when predicting future outcomes.

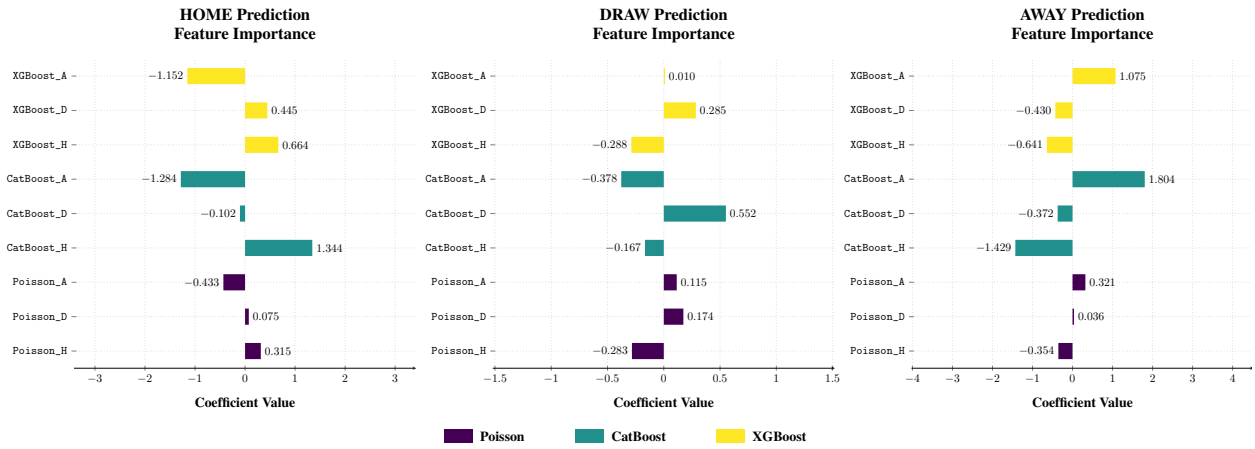


Figure 4: Meta-Learner coefficients by outcome class

4.6.1 Data Partitioning

The dataset was divided into three sequential subsets:

- **Training Set (70%):** Used for feature learning and base model parameter estimation.
- **Validation Set (15%):** Used for hyperparameter tuning, early stopping of gradient boosting models, and training the stacking meta-learner. All hyperparameters were selected based on validation performance trends rather than test-set optimisation.
- **Test Set (15%):** A strictly sequestered hold-out set used exclusively for the final performance evaluation.

4.6.2 Leakage Prevention

Rigorous measures were taken to eliminate all forms of data leakage. All engineered temporal features (e.g., rolling averages, form indicators, Elo trends) were computed using a `.shift(1)` operation to ensure that only information from matches occurring strictly before the prediction target was included.

In addition, all contemporaneous match statistics unavailable at prediction time (e.g., shots on target, possession, fouls) were removed from the feature set prior to training. These safeguards ensure that the reported performance reflects genuine predictive capability rather than artefacts of information leakage.

4.6.3 Out-of-Fold Prediction

Leakage Prevention
Predictions on training data are optimistically biased if used as downstream features, this causes data leakage. Out-of-fold (OOF) prediction ensures every training sample receives a prediction from a model that never observed it. Using TimeSeriesSplit with 5 folds, each fold trains on preceding data and predicts on the subsequent segment; these are concatenated to form uncontaminated training predictions.

OOF is applied in two places: (1) **P(Close) feature generation**—the Closeness Classifier (Section 4.3.1) generates `p_close` via OOF, ensuring CatBoost and XGBoost are not indirectly exposed to future outcomes; (2) **Base model stacking**—CatBoost and XGBoost predictions for meta-learner training are generated via OOF, preventing the meta-learner from learning on inflated base model accuracy.

Regularisation Effect
OOF predictions are inherently noisier since they come from models trained on limited data subsets. Downstream models

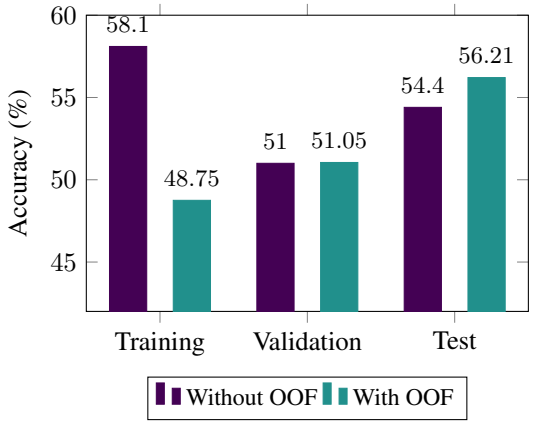


Figure 5: Effect of Out-of-Fold prediction on ensemble accuracy. Training accuracy decreases while test accuracy improves, demonstrating successful regularisation.

thus learn with prediction quality matching inference conditions.
This reduces training accuracy but improves generalisation, preventing over-reliance on derived features.

4.6.4 Post-Deployment Simulation

Beyond the formal test set, we conducted a "live" simulation on 20 matches from the 2025/2026 EPL season to assess model robustness on unseen future data. This evaluation represents the most realistic usage scenario, as it tests the complete pipeline on data that was unavailable during model development.

The ensemble achieved an accuracy of **55%**. While the sample size is small and inherently noisy, this performance aligns closely with validation metrics, providing additional evidence that the model generalises beyond historical data and is not overfitting to training-era dynamics.

Table 1: Real-World Test Prediction Results (25/26 Season)

Home	Away	Pred	Act	Res	Home	Away	Pred	Act	Res
Bournemouth	Everton	H	A	✗	Aston Villa	Arsenal	H	H	✓
Fulham	Man City	A	A	✓	Bournemouth	Chelsea	A	D	✗
Newcastle	Tottenham	H	D	✗	Everton	Nott'm Forest	H	H	✓
Arsenal	Brentford	H	H	✓	Man City	Sunderland	H	H	✓
Brighton	Aston Villa	A	A	✓	Newcastle	Burnley	H	H	✓
Burnley	Crystal Palace	A	A	✓	Tottenham	Brentford	A	H	✗
Wolves	Nott'm Forest	A	A	✓	Leeds	Liverpool	A	D	✗
Leeds	Chelsea	A	H	✗	Brighton	West Ham	A	D	✗
Liverpool	Sunderland	H	D	✗	Fulham	Crystal Palace	A	A	✓
Man United	West Ham	A	D	✗	Wolves	Man United	A	A	✓

Accuracy: 55.00% (11/20 correct)
Note: Predictions generated using the full Ensemble model on unseen future data. This test set is small and results are inherently noisy.

4.7 Model Selection Rationale

The final ensemble configuration was selected based on a combination of validation performance, calibration quality, and robustness across time-based splits. While individual models exhibited strengths in specific regimes, the stacked ensemble consistently outperformed its components by exploiting their complementary error profiles.

5 Results

Overall we achieved a 56.21% accuracy rate on the test data with our model, beating the bookies rate of 53% [1]. Figure 6 shows our precision, recall and F1-scores [14] for Home, Draw, and Away predictions.

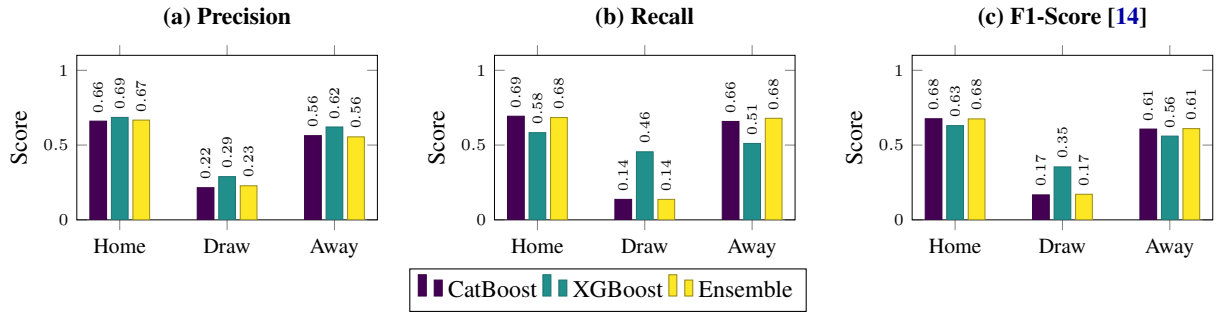


Figure 6: Model Performance Comparison by Metric

5.1 Model Comparison

Figure 6 reveals distinct behavioural profiles across base models. CatBoost achieves the highest home win recall (0.69) and strong away detection (0.66), but severely underpredicts draws (46/336, 13.7% recall). XGBoost's draw-boost weighting triples draw recall to 45.5% (153/336), though at the cost of reduced home and away detection. The ensemble

balances these trade-offs, matching CatBoost's home/away performance while draw prediction remains constrained by the inherent difficulty of the class.

5.2 Out-of-Fold Regularisation Impact

Training accuracy decreased from 58.10% to 48.75% when OOF predictions replaced direct predictions for meta-learner training, while test accuracy improved from 54.40% to 56.21%. This test improvement confirms OOF successfully prevented the meta-learner from overfitting to optimistically biased base model outputs.

5.3 Confusion Matrix Analysis

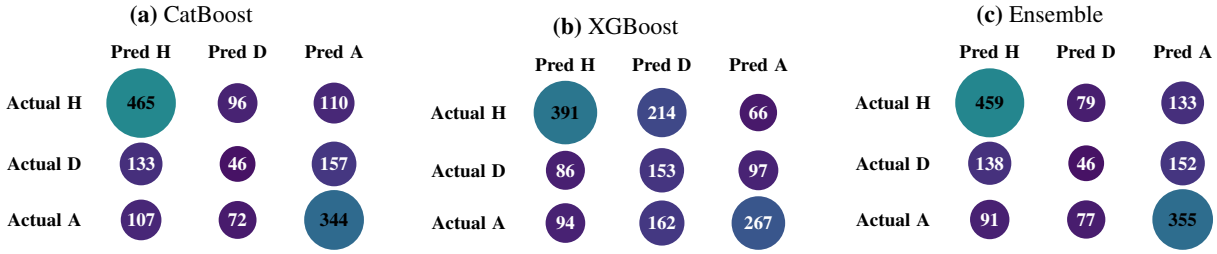


Figure 7: Comparison of Confusion Matrices

Figure 7 reveals distinct model behaviours. CatBoost achieves highest home win detection (465/671, 69.3% recall) but severely underpredicts draws (46/336, 13.7% recall), misclassifying draws roughly equally as home wins (133) and away wins (157). XGBoost's draw-boost weighting triples draw recall to 45.5% (153/336), demonstrating effective cost-sensitive learning for the minority class.

The ensemble (Figure 7c) shows draw recall remained at 13.7% (46/336). Coefficient analysis (Figure 4) revealed the meta-learner assigned higher weights to CatBoost's draw probability than XGBoost's, despite XGBoost's superior draw detection. This motivated the draw prediction override (Section 4.5.4), routing draw classification exclusively through XGBoost.

6 Final Predictions on Test Set

Table 2: Upcoming Match Predictions for January 31, 2026

Date	Home Team	Away Team	Prediction
31 Jan 26	Leeds	Arsenal	A
31 Jan 26	Liverpool	Newcastle	H
31 Jan 26	Tottenham	Man City	A
31 Jan 26	Wolves	Bournemouth	A
31 Jan 26	Aston Villa	Brentford	H
31 Jan 26	Brighton	Everton	A
31 Jan 26	Chelsea	West Ham	H
31 Jan 26	Man United	Fulham	A
31 Jan 26	Sunderland	Burnley	A
31 Jan 26	Nottingham Forest	Crystal Palace	A

7 Conclusion

7.1 Summary of Approach

This project developed a stacking ensemble for EPL match prediction, achieving approximately 56% accuracy on the held-out test set, exceeding the 53% bookie benchmark [1]. The ensemble combined complementary paradigms: a time-aware Poisson model grounded in football theory, and CatBoost/XGBoost classifiers trained on 70+ engineered features including Elo ratings, xG metrics, and head-to-head statistics. An auxiliary closeness model provided learned match-tightness probabilities as additional input features, capturing structural patterns associated with closely contested fixtures.

7.2 Limitations

Despite these advances, draw prediction remains fundamentally difficult; our ensemble achieved only 14% draw recall. Draws occur when evenly-matched teams both fail to capitalise, a scenario poorly captured by aggregated strength differentials. Additionally, certain advanced features (e.g., long passing metrics) were only available for recent seasons, limiting the model's ability to learn from older historical patterns.

7.3 Future Work

Future work could explore several directions: modelling manager-specific tactical patterns, integrating player-level availability data, and developing more robust draw prediction mechanisms; potentially through draw-specific auxiliary models or asymmetric loss functions that further penalise draw misclassification. Additionally, recurrent architectures that model team form as sequential state could better capture momentum dynamics than rolling aggregates.

References

[1] D. Dr. Hosseini, "COMP0036 Group Coursework Brief," 10 2025.

[2] "epl-training - Google Sheets." [Online]. Available: https://docs.google.com/spreadsheets/d/e/2PACX-1vSZHBZihHdDS_mjw6eQD-2KmH-sQX2sp67bMwl88Z1bVuOetBUrli3EgWa_u6chnj978iMBw0isMuRJ/pub

[3] "epl-stats - Google Sheets." [Online]. Available: https://docs.google.com/spreadsheets/d/e/2PACX-1vSa19Qm_VdDpu93RpRuDopVMRJ288DEFFya7hQonlChDlL2tsJeyuQU-F2LWPJMFgdipVmXE9ccNela/pub

[4] "epl-elo - Google Sheets." [Online]. Available: https://docs.google.com/spreadsheets/d/e/2PACX-1vRCILgwHYY9gDGPYztr-i-KYLYHABzz6JEvT-cAVyWh-GVh2Kf_xZVoQZLwPNBvh-9aMZLrE3yvg1d3/pub

[5] "Free Sports API for Live Scores, Fixtures & Player Data | SportDB.dev." [Online]. Available: https://sportdb.dev/

[6] "Football Elo Ratings | ClubELO." [Online]. Available: http://clubelo.com/

[7] B. Holmes and I. G. McHale, "Forecasting football match results using a player rating based model," *International Journal of Forecasting*, vol. 40, no. 1, pp. 302–312, 2024. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S016920702300033X

[8] I. Cleland, A. Moore, H. Wang, D. Glass, J. Rafferty, R. Bond, J. Wallace, and P. Hassard, "Predicting Football Match Outcomes Using Event Data and Machine Learning Algorithms Predicting Football Match Outcomes Using Event Data and Machine Learning Algorithms. In H Predicting Football Match Outcomes using Event Data and Machine Learning Algorit. . . ." [Online]. Available: https://creativecommons.org/share-your-work/cclicenses/.

[9] U. Haruna, J. Maitama, M. Mohammed, and R. Raj, *Predicting the Outcomes of Football Matches Using Machine Learning Approach*, 01 2022, pp. 92–104.

[10] T. Chen and C. Guestrin, "Xgboost: a scalable tree boosting system," *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining - KDD '16*, vol. 1, p. 785–794, 08 2016.

[11] A. V. Dorogush, V. Ershov, and A. G. Yandex, "Catboost: gradient boosting with categorical features support," 10 2018. [Online]. Available: https://arxiv.org/pdf/1810.11363

[12] V. Braverman, H. Lang, E. Ullah, and S. Zhou, "Improved Algorithms for Time Decay Streams," 2019. [Online]. Available: https://arxiv.org/pdf/1907.07574v1

[13] K. Wada and T. Kurosawa, "The naïve estimator of a poisson regression model with measurement errors," 2022. [Online]. Available: https://arxiv.org/abs/2205.05254

[14] C. J. V. Rijsbergen, *Information Retrieval*, 2nd ed. Butterworth-Heinemann, 1979.

AI Declaration

We acknowledge the use of Claude (version Claude Sonnet 4.5, Anthropic, `https://claude.ai/`) to help gather information about the current state of the field. Gemini (version 3.5 Pro and 3.5 Flash, Google, `https://gemini.google.com/`) was used in an assistive role to help proof-read written content as well as fix mistakes when constructing LaTeX diagrams. All AI usage was in accordance with Category 2 of UCL's classification of AI assistance.